CS1540 Algorithmic design

Steven Chaplick, David Mestel

2025 exam model solutions

1. (a) **Main Idea (greedy rule):** iteratively pick the fueling station that is the furthest from the current position without running out of fuel.

   Variables:

   - stops: List of integers storing the indices of stations where we will stop for fuel. This is the k-partial solution for our greedy rule where k represents the first k stations along the corridor.
   - p: double to denote the position of the ship. Initially p = 0.

   Code:

```
for i := 1 to n-1
   if d_{i+1} > p+t: // if the next station is too far,
                     //stop at station i
      stops.add(i);
      p := d_i;
   end if
end for
if d* > p+t: //if the end of the road is not reachable,
            //then we should stop at the last station.
   stops.add(i);
end if
return stops;
```

   (b) Let stops be the stations chosen by the greedy approach from 1a.

   Let OPT be an optimal selection of stations that has as many stations in common with stops as possible.

   If OPT == stops, greedy is optimal and we are done. Otherwise stops is not optimal and must be different from OPT.

   Let j be the index of the first station (the station with the smallest index) where OPT and stops disagree, and let k be the last station before j in both OPT and stops.

   This gives us two cases.

   - Case 1: $j \in$ OPT, $j \notin$ stops.
     Let j^* be the next station added to stops after iteration j. By construction of our greedy solution, j^* can be reached from stop k and $d_{j^*} \geq d_j$. Consider the set S of stops obtained by replacing j with j^*. Let l be the stop in OPT that occurs after j. Now, since l can be reached from j and $d_{j^*} \geq d_j$, l can also be reached from j^*.
     Thus S is an optimal solution since it is valid and uses the same number of stops as OPT. However S has more commonality stops: This contradicts the maximum commonality of OPT and stops.
   - Case 2: $j \notin$ OPT, $j \in$ stops.

Since j is in stops, there are no stations with index larger than j and distance larger than d_j that can be reached from k. Thus, the next station j^* in OPT must have the same distance as j. Here we can create a new optimal solution by replacing j^* with j, but this contradicts the maximum commonality of OPT and stops.

In both cases we contradict that the chosen OPT and stops are different. Thus, stops must be an optimal solution.

(Remark regarding Case 2: the implementation in 1a actually ensures that we would pick the last station with this distance, but the argument here does not rely on that)

2. (a) Compare n with $n^{\log_2 6}$. $\log_2 6 > 2 > 1$ so we are in the case where only the recursive work matters and the solution is $T(n) = \Theta(n^{\log_2 6})$.
   (b) Compare 2^n with $n^{\log_2 8} = n^3$. 2^n is way bigger [ideally: $2^n = \Omega(n^d)$ for some $d > 3$, e.g. $d = 3.1$, but not required for full credit], so only the recombination work matters, solution is $T(n) = \Theta(2^n)$.
   (c) Compare $\sqrt{n} = n^{1/2}$ with $n^{\log_4 2} = n^{1/2}$, so we are in the case where both matter, solution is $T(n) = \Theta(\sqrt{n} \log n)$.

3. Disclaimer: This solution runs in $O(n)$ time, but for full credit, $O(n^2)$ time would still be ok. In the $O(n^2)$ algorithm, the table has $O(n^2)$ entries, but each entry can still be computed in $O(1)$ time.

   - Step1:
     Let OPT[j] be the sum of the subarray ending in index j with largest sum. That is,
     $$\text{OPT}[j] = \max_i(\textstyle\sum_{p=i}^{j} X[p]).$$
     In this way, the table captures the best solution that can be obtained when insisting that the subarray uses and ends in index j.
     Note that, with this table, the value of the best subarray is: $\max_j$ OPT[j].
   - Step 2:
     The recursive structure is that the best solution when using and ending in position j can be found by either:
     - extending the previous solution: taking the best solution ending in and using position $j-1$ and adding $X[j]$ to it. or
     - dropping the previous solution: using only the value $X[j]$, eg, when OPT[$j-1$] ≤ 0 we do not want to extend it.

     With this in mind, the recurrence is: `OPT[j] = max {X[j], X[j] + OPT[j-1]}`
   - Step 3:

```
Initialize OPT[j] = -infinity for each j.
\\fill in the table
for j := 0 to n-1
   OPT[j] := max {X[j], X[j] + OPT[j-1] }.
end for
\\Compute the optimal value and save the end index of the best subarray.
bestSum := -inf
endSubarray := 0
for j := 0 to n-1
   if bestSum < OPT[j]
      bestSum := OPT[j]
      endSubarray := j
   end if
end for
```

Runtime: The first loop has n iterations, and each uses $O(1)$ steps. The second loop has the same timing. Thus, the algorithm runs in $O(n)$ time in total.

- Step 4:
  The explanation of correctness of the recurrence is given in Step 2. So, here we only argue about the correctness of the reconstruction. What remains after Step3 is simply to find the start index of the best subarray. Based on our recurrence, the subarray extends one step earlier precisely when
  `OPT[j] == OPT[j-1]+X[j]`
  Code:

```
startSubarray := endSubarray; \\initialize start=end
while OPT[startSubarray] == OPT[startSubarray-1]+X[startSubarray]
   startSubarray-- ;
end while
```

  Runtime: Since the loop makes at most n iterations, and each iteration uses $O(1)$ steps, the time here is again $O(n)$.

4. (a) INPUT: integers n, B, T, $n-1$ tuples of integers $(s_1, e_1, b_1, c_1), \ldots, (s_{n-1}, e_{n-1}, b_{n-1}, c_{n-1})$ and integers $N_1, \ldots, N_k$.
   OUTPUT: 1 [or YES or similar] if for this set of roads (where the starting and ending points, travel time and upgrade cost of road i is given by (s_i, e_i, b_i, b_i)) there exists a set of road upgrades of total cost at most B, leading to $T_{N_1} + \ldots + T_{N_k} \leq T$ (where T_{N_i} are defined as in the question.
   0 [or NO etc.] otherwise.

   (b) Be NP-hard and be in NP.

   (c) We will reduce from 0-1-knapsack. Given a knapsack instance l with o objects with weights and values (w_i, v_i), capacity C and target value V, define a Roads instance $f(l)$ with $n = o+1$ towns and one noble with $N_1 = o+1$. Let road i connect town i to town $i+1$, and have $c_i = w_i$ and $b_i = 2v_i$. Let $B = C$ and $T = (\sum_{i=1}^{o} 2v_i) - V$.

   We now show that $f(l)$ is a YES instance of Roads if and only if l is a YES instance of 0-1-knapsack. Note that if we pick a set S of roads to upgrade then the total time is given by

$$\sum_{i=1}^{o} \begin{cases} 2v_i \text{ if } i \notin S \\ v_i \text{ if } i \in S \end{cases} = \left(\sum_{i=1}^{o} 2v_i\right) - \left(\sum_{i \in S} v_i\right)$$

   which is $\leq T$ if and only if $\sum_{i \in S} v_i \geq V$. The cost of upgrading S is $\sum_{i \in S} c_i = \sum_{i \in S} w_i$. Hence $f(l)$ has a set of roads of total cost at most B achieving upgrade time at most T if and only if l has a set of objects of total weight at most C and total value at least V, as required.
   [note we could also have done this by having each town connected directly to the capital and a noble at each town. Note also this is very similar to the 'weg afgesloten' problem from the assignment sheets.]

   (d) Variables: u_i, T_j, for $1 \leq i \leq n-1$ and $1 \leq j \leq n$
   Minimise $T_{N_1} + T_{N_2} + \ldots + T_{N_k}$
   Subject to:
   $u_1 c_1 + u_2 c_2 + \ldots + u_{n-1} c_{n-1} \leq B$
   $(1 - u_{i_1}/2) b_{i_1} + (1 - u_{i_2}/2) b_{i_2} + \ldots + (1 - u_{i_d}/2) b_{i_d} - T_j \leq 0$ for each j, where $r_{i_1} r_{i_2} \ldots r_{i_d}$ is the unique path from the capital to town j.
   $0 \leq u_i \leq 1$ for each i

each $u_i \in \mathbb{Z}$ [i.e. is an integer]

[note these are all linear in the variables of our program, $T_1, \ldots T_n, u_1, \ldots, u_{n-1}$]

(e) The fractional relaxation means dropping the integrality constraints on the u_i, giving an LP which can be solved in polynomial time. Since the optimum of the relaxation is at least as good as the optimum of the original problem, this gives an efficiently-computable bound on how well a partial solution can be completed to a full solution, which we can use in a branch-and-bound algorithm.

[note that backtracking and branch-and-bound were covered in more detail this year than last year so there could be questions asking e.g. to actually write a backtrack or branch-and-bound algorithm]

5. Not relevant this year.